

SYSTEM ROLE:

You are an expert robotics/embedded-systems engineer specializing in Raspberry Pi animatronics: OpenCV face tracking, audio amplitude detection, servo control, LED PWM, and behavior-driven state machines.

TASK:

Generate a complete, runnable Python 3.11+ codebase for a full-size animatronic Halloween skeleton, split across the modules below. Output all files in one response with clear file-path headers. No placeholders, no TODOs — every function must have working example logic (using mocked hardware where real hardware is unavailable).

HARDWARE ASSUMPTIONS (state these as comments in config.py)

- Raspberry Pi 4/5, Raspberry Pi Camera Module (picamera2)
- MG996R servo ×2 (head pan, jaw) via PWM (gpiozero or pigpio — pick one and use it consistently)
- USB microphone or I2S mic, read via sounddevice or pyaudio
- Common-cathode RGB or amber/red LED matrix for eyes, PWM-driven
- Optional: knock sensor (piezo/vibration) on a GPIO input pin, proximity sensor (e.g., HC-SR04 or VL53L0X) for the <2ft trigger
- Target 20–30 FPS control loop; assume no GPU acceleration

MODULE SPECIFICATIONS

config.py

- Dataclass or constants for: GPIO pin assignments (head servo, jaw servo, LED(s), knock sensor, proximity sensor), servo angle limits, audio RMS thresholds (ambient/loud-noise), face-tracking gain/smoothing constants, proximity threshold, mode-timeout duration, audio file paths, TTS voice params.

vision.py

- `FaceTracker` class: init Pi Camera (or `cv2.VideoCapture` fallback), Haar cascade or DNN face detection, compute horizontal pixel offset from frame center, convert to smoothed servo angle (exponential moving average), estimate relative distance from bounding-box size for LED brightness scaling.

audio.py

- `AudioInput` class: read mic amplitude (RMS) in a background thread, expose latest value non-blocking, map amplitude → jaw servo angle (linear or log-scaled), detect loud-noise threshold crossing (mode-switch trigger).

- ``AudioOutput`` class: non-blocking playback of pre-recorded WAV/MP3 (use ``threading`` or ``asyncio`` + a playback queue — pick one and justify briefly), and TTS generation/playback (pyttsx3 offline, with a stub for cloud TTS).

`servo.py`

- ``ServoController`` class: set angle with clamping, easing functions (e.g., ease-in-out) for smooth transitions, separate methods for head and jaw servos.
- Mock backend (prints/logs angle changes) auto-selected when ``gpiozero``/RPi hardware isn't detected, so the whole system runs on a dev machine.

`led.py`

- ``EyeController`` class: PWM brightness control, flicker pattern generator (randomized or scripted), color switch between amber (Host) and red (Trickster).

`behavior.py`

- ``BehaviorManager`` state machine with two states: ``HOST`` and ``TRICKSTER``.
- Constructed via dependency injection: takes ``FaceTracker``, ``AudioInput``, ``AudioOutput``, ``ServoController``, ``EyeController`` as constructor args (no internal instantiation).
- Trickster triggers: loud noise, knock sensor, proximity < 2ft.
- Auto-revert to Host after a configurable timeout of no triggers.
- Each mode has distinct: tracking smoothness, LED color/flicker profile, and a set of example line triggers (both pre-recorded file keys and TTS strings).

`main.py`

- Instantiate all modules (real or mock, auto-detected), inject into ``BehaviorManager``, run a non-blocking main loop (fixed tick rate, no blocking calls) that: reads sensors → updates behavior state → updates servos/LEDs → fires audio as needed.
- Include a ``--test`` CLI flag that runs a short simulated session using mock hardware and asserts basic state transitions (simple test harness, not a full pytest suite unless you'd prefer one — default to a lightweight inline harness).

CODING STANDARDS

- Python 3.11+, full type hints, docstrings (Google or NumPy style, pick one) on every public class/method.
- One hardware abstraction layer per module so real and mock backends share an interface (e.g., ABC or Protocol).
- No blocking sleeps in the main loop; use `threading`/`asyncio` for audio and sensor polling.
- Comment every hardware-specific assumption inline, not just in `config.py`.

OUTPUT FORMAT

1. Project tree first (brief).
2. Each file in full, separated by ``# ===== path/filename.py =====`` headers.
3. End with numbered setup/run instructions for actual Raspberry Pi deployment (dependencies, GPIO wiring notes, permissions, autostart).

Do not ask clarifying questions — resolve ambiguity using the defaults stated above and note any additional assumptions as a short bullet list at the very end.